빅데이터기초 중간고사 핵심 정리

2026-2학기 · 이청용 교수 · 시험 범위 3~7주차 + 실습 · 중간고사 10/19(월) 18:30

> **이 정리의 기준**: 보내주신 요약 PDF 2개를 강의 PPT·노트북과 하나씩 대조해서, **강의에 나온 내용만** 남기고 빠진 핵심 내용(제어문, `is_even()`, `while`, 배열 분할, 유니버설 함수, 실습 3 등)을 채웠습니다. 출력 예시는 모두 직접 실행해서 확인한 값입니다.
>
> 실습 코드 읽기·출력 예측으로 나오기 좋은 부분 함정 틀리기 쉬운 부분 비교 객관식 단골 비교

1. 3주차 · 기본 데이터 타입 (숫자와 문자열)

객체와 변수

- Python의 모든 데이터(숫자, 문자열, 자료구조, 함수 등)는 **객체**이고, 변수는 객체를 **가리키는 이름**이다.
- 같은 변수에 새 값을 할당하면 기존 객체를 바꾸는 게 아니라 **새 객체를 가리키게** 된다.

함정 변수 이름 규칙

규칙	예시
사용 가능 문자	소문자, 대문자, 숫자, 언더스코어 `_` → `name`, `score_95`, `_abc` 가능
숫자로 시작 불가	`1name`, `95score`, `1_`, `1st_student` → 오류
대소문자 구분	`Data` 와 `data` 는 다른 변수
예약어 사용 불가	`if`, `for`, `class`, `def`, `return`, `True`, `False`

숫자형과 연산자

정수 `int` (`123`), 실수 `float` (`-1.23`), 지수 표현(`4.24e10` = 4.24×10^{10}, `4.24e-10` = 4.24×10^{-10})

연산자	의미	예시 → 결과	포인트
`/`	나눗셈	`7 / 2` → `3.5`	나누어떨어져도 실수: `4 / 2` → `2.0`
`//`	몫	`7 // 2` → `3`	소수점 이하 버림
`%`	나머지	`7 % 3` → `1`	짝수·홀수 판별에 사용 (7주차 `is_even`)
`**`	거듭제곱	`3 ** 4` → `81`	

곱셈이 덧셈보다 먼저: `a = 5, b = 10` 일 때 `a + b - a * b` → `15 - 50` = **`-35`**

형변환

함수	예시 → 결과
`int()`	`int(True)` → `1`, `int(False)` → `0`, `int(98.7)` → `98` (반올림 아님, 버림), `int("89")` → `89`
`float()`	`float(10)` → `10.0`, `float(True)` → `1.0`, `float("98.7")` → `98.7`
`str()`	`str(98.6)` → `'98.6'`, `str(1.0e4)` → `'10000.0'`, `str(True)` → `'True'`

문자열 기본

- 작은따옴표 `'`, 큰따옴표 `"`, 여러 줄은 `'''` 또는 `"""`
- 이스케이프 문자: `\n` 줄바꿈, `\t` 탭
- `+`로 결합 (공백은 자동으로 안 들어감): `"Hello" + " " + "Python"` → `Hello Python`
- 리터럴 문자열은 나란히 쓰면 자동 결합: `"Gildong" " " "Hong"` → `Gildong Hong`
- `print()`에 여러 값을 쓰면 사이에 공백이 자동으로 들어감: `print("Name:", "Alice")` → `Name: Alice`
- `*`로 반복: `"Python" * 3` → `PythonPythonPython`, `"-" * 20`

실습 인덱싱과 슬라이싱 `[start:end:step]`

```
string = "Python"
string[0]    # 'P'      왼쪽은 0부터
string[-1]   # 'n'      오른쪽은 -1부터
string[-6]   # 'P'
string[10]   # IndexError (범위 초과)
string[0:2]  # 'Py'     end는 포함 안 함
string[2:]   # 'thon'
string[::-1] # 'nohtyP' 역순
"abcdefg"[::2]  # 'aceg'
```

함정 **문자열은 불변(immutable)**이라 한 글자만 바꿀 수 없다.

```
name = "Henny"
name[0] = "P"           # TypeError
name.replace("H", "P")  # 'Penny'  (새 문자열을 반환, name은 그대로)
"P" + name[1:]          # 'Penny'  (슬라이싱 + 결합)
```

문자열 메서드

메서드	예시 → 결과	포인트
`split(sep)`	`"apple,banana".split(',')` → `['apple', 'banana']`	인자 없으면 공백(스페이스·탭·줄바꿈) 기준. 결과는 **리스트**
`join(list)`	`', '.join(['Hello', 'Python'])` → `'Hello, Python'`	구분자 문자열에서 호출
`replace(old, new, n)`	`"apple, apple".replace("apple", "orange", 1)` → `'orange, apple'`	세 번째 인자 = 바꿀 횟수
`len()`	`len("Hello World")` → `11`	공백도 한 글자
`find` / `rfind`	`"apple banana apple".find("apple")` → `0`, `rfind` → `13`	첫 위치 / 마지막 위치. 없으면 **`-1`** (오류 아님)
`count(sub)`	`"apple banana apple".count("apple")` → `2`	
`upper` / `lower`	`"hello python".upper()` → `'HELLO PYTHON'`	
`capitalize` / `title`	`capitalize()` → `'Hello python'`, `title()` → `'Hello Python'`	문장 첫 글자만 / 단어마다
`format()`	`"{0}님, {1}살".format("Alice", 20)` → `'Alice님, 20살'`	`{0}`, `{1}`은 위치 인덱스

2. 4~5주차 · 데이터 구조 (리스트, 튜플, 딕셔너리, 집합)

리스트 `[ ]` — 순서 있음, 수정 가능, 중복 허용

생성: `[10, 20, 30]`, 빈 리스트 `[]` 또는 `list()`. 여러 타입을 섞을 수 있음 `[1, 3.14, "Python", True]`. 인덱싱·슬라이싱은 문자열과 같음.

추가	
`append(x)`	맨 끝에 1개 추가
`insert(i, x)`	i 위치에 추가. **범위 밖이면 맨 끝**에 추가
`extend(lst)`, `+=`	다른 리스트를 이어 붙임
`lst[i] = x`	기존 위치 값 수정 (유효한 인덱스만)

비교 삭제	
`del lst[i]`	**위치**로 삭제
`remove(x)`	**값**으로 삭제. 같은 값이 여러 개면 **첫 번째 하나만**
`pop(i)`	삭제하면서 **그 값을 반환**. 인자 없으면 마지막 값
`clear()`	(딕셔너리) 전체 비우기

```
numbers = [10, 20, 30]
numbers.insert(1, 99)    # [10, 99, 20, 30]
numbers.insert(10, 50)   # [10, 99, 20, 30, 50]  범위 밖 → 맨 끝

numbers = [10, 20, 30, 40, 50]
removed = numbers.pop(2) # removed = 30, numbers = [10, 20, 40, 50]
numbers.pop()            # 50 반환, numbers = [10, 20, 40]

dup = [1, 2, 2, 3]
dup.remove(2)            # [1, 2, 3]  첫 번째 2만 삭제
```

리스트 확인

- `index(x)` : 첫 번째로 나오는 위치. `[10, 20, 30, 20, 40].index(20)` → `1`. 없으면 `ValueError`
- `len(lst)` : 개수, `count(x)` : x가 나온 횟수

함정 **`sort()` vs `sorted()`**

	`lst.sort()`	`sorted(lst)`
원본	원본 자체를 정렬 (바뀜)	원본 유지
반환값	**`None`**	정렬된 **새 리스트**
내림차순	`reverse=True` 인자 추가 · 숫자는 오름차순, 문자열은 알파벳순	

```
a = [3, 1, 2]
print(a.sort())   # None  ← 함정!
print(a)          # [1, 2, 3]
```

튜플 `( )` — 순서 있음, 수정 불가(불변)

- "상수의 리스트". 값 추가·삭제·수정이 안 되고, `append()`, `insert()` 같은 메서드가 없다.
- 괄호는 필수가 아님: `t2 = 40, 50, 60` 도 튜플
- 함정 원소가 1개면 **콤마 필수**: `(70,)` 은 튜플, `(70)` 은 그냥 정수 `int`

- 수정 시도 `t1[0] = 99` → `TypeError`

```
a, b, c = (10, 20, 30)   # 튜플 언패킹 → a=10, b=20, c=30
x, y = 1, 2
x, y = y, x              # 값 교환 (임시 변수 없이) → x=2, y=1
```

튜플을 쓰는 이유: 메모리를 덜 씀 · 불변이라 데이터가 안전함 · **딕셔너리의 키로 사용 가능** · 네임드 튜플로 객체 대신 사용 · 함수 인자가 튜플로 전달됨(`*args`)

딕셔너리 `{키: 값}` — 키로 값을 관리

- 인덱스(위치)로 접근 불가, **키**로 접근. 키는 **중복 불가** (같은 키에 다시 넣으면 마지막 값으로 덮어씀)
- 빈 딕셔너리: `{}` 또는 `dict()`
- `d["key"] = 값` : 키가 있으면 **수정**, 없으면 **추가**
- `update()` : 다른 딕셔너리나 `키=값` 인자로 병합 (강의 자료의 `updated()` 는 오타)
- `del d["key"]` : 특정 키 삭제 (없는 키면 `KeyError`) · `d.clear()` : 전체 비움 → `{}`
- `"name" in d` : **키**가 있는지 확인 → `True` / `False` , `not in` 도 가능
- `keys()` → `dict_keys([...])` , `values()` → `dict_values([...])` , `items()` → `(키, 값)` 튜플 쌍. `list()` 로 변환 가능

```
student = {"name": "Alice", "age": 21}
student.update({"age": 22, "major": "CS"})
# {'name': 'Alice', 'age': 22, 'major': 'CS'}
student.update(grade="A", name="Bob")
# {'name': 'Bob', 'age': 22, 'major': 'CS', 'grade': 'A'}
```

함정 **대괄호 조회 vs `get()`**

```
student = {"name": "Alice", "age": 21}
student["grade"]              # KeyError
student.get("grade")          # None   (오류 안 남)
student.get("grade", "N/A")   # 'N/A'  (기본값 지정)
```

집합 `{ }` — 순서 없음, 중복 불가

- 값 없이 키만 있는 딕셔너리와 비슷. 주로 **포함 여부** 판단에 사용
- `{1, 2, 3, 3}` → `{1, 2, 3}` , `set([1, 2, 2, 3])` → `{1, 2, 3}`
- **함정** 빈 집합은 반드시 `set()` . `{}` 는 **빈 딕셔너리**

`a = {1, 2, 3, 4}` , `b = {3, 4, 5, 6}`	연산자	메서드	결과
교집합 (둘 다 있는 값)	`a & b`	`intersection()`	`{3, 4}`
합집합 (모든 값, 중복 제거)	`a | b`	`union()`	`{1, 2, 3, 4, 5, 6}`
차집합 (a에만 있는 값)	`a - b`	`difference()`	`{1, 2}`
대칭 차집합 (한쪽에만 있는 값)	`a ^ b`	`symmetric_difference()`	`{1, 2, 5, 6}`

비교 4대 자료구조 한눈에 보기

구분	기호	수정	순서·인덱스	중복	빈 객체
리스트	`[ ]`	가능	있음	허용	`[]`, `list()`
튜플	`( )`	**불가**	있음	허용	`()`, `tuple()`
딕셔너리	`{키: 값}`	가능	인덱스 없음 (키로 접근)	키 중복 불가	`{}`, `dict()`
집합	`{ }`	가능	없음	**불가**	`set()` 만

딕셔너리 순서: 강의 자료는 "순서를 따르지 않음"이라고 하지만, 실제로 Python 3.7부터는 넣은 순서가 유지됩니다(출력 순서도 그대로). 시험에서는 강의 표현대로 "인덱스로 접근할 수 없다"를 기억하세요.

3. 6주차 · NumPy 다차원 배열

NumPy 개요

- Python 수치 계산의 핵심 패키지. 대용량 배열을 효율적으로 다루도록 설계되어 **리스트보다 훨씬 빠름** (강의 예시: 100만 개 ×2 연산, NumPy 약 31ms vs 리스트 약 628ms)
- NumPy 자체는 모델링 기능이 없고, 배열 연산을 바탕으로 Pandas 같은 고수준 도구를 활용
- 설치 `pip install numpy` / `conda install numpy`, 불러오기 `import numpy as np`

ndarray 만들기와 4대 속성

- `np.array()` 로 생성. 리스트와 달리 **모든 원소가 같은 타입**이어야 하고, 섞이면 상위 타입으로 자동 변환 (`[1, 2.5]` → `float64`)

```
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8]])
arr_2d.ndim    # 2        차원 수
arr_2d.shape   # (2, 4)   각 차원 크기 (튜플)
arr_2d.size    # 8        전체 원소 수 (shape를 곱한 값)
arr_2d.dtype   # int64    원소 타입
```

배열 생성 함수

함수	설명
`array`	리스트·튜플 등을 ndarray로 변환. dtype을 안 주면 추론
`arange`	`range` 와 비슷하지만 ndarray 반환. 끝값 미포함 (`np.arange(1, 10, 2)` → `[1 3 5 7 9]`)
`zeros` / `ones`	0 / 1로 채움. **기본 타입은 `float64`** 라서 `0.`, `1.` 로 출력
`full`	지정한 값으로 채움
`linspace`	일정한 간격으로 생성. 끝값 포함 (`np.linspace(0, 1, 5)` → `[0. 0.25 0.5 0.75 1.]`)
`eye`, `identity`	N × N 단위행렬

```
np.zeros(10)       # [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]   1차원 10개
np.ones((3, 6))    # 3행 6열, 모두 1.   ← 모양은 튜플로 전달
```

np.random	설명
seed	난수 시드 지정
shuffle	순서 섞기
rand	균등분포 표본
randint	범위 안 임의 정수. np.random.randint(20, size=(2, 3)) → 0 이상 20 **미만**, 2×3
randn	평균 0, 표준편차 1 정규분포
normal	정규분포. np.random.normal(size=(4, 4))

실습 배열 산술 연산 (벡터화)

- 반복문 없이 **원소별로** 일괄 계산 = 벡터화(vectorization)
- 같은 크기 배열끼리: `arr * arr` (원소별 곱), `arr - arr` (모두 0)
- 스칼라와 연산: 모든 원소에 적용. `1 / arr` (역수), `arr * 0.5`
- 비교 연산: **논리값 배열** 반환. `arr2 > arr1` → `[[True False ...]]`

실습 인덱싱과 슬라이싱

```
x2[0, 0]       # 다차원은 콤마로 구분해서 접근
x2[0, 0] = 12  # 값 수정도 가능
x1[-1]         # 마지막 원소

x = np.arange(10)   # [0 1 2 3 4 5 6 7 8 9]
x[5:]      # [5 6 7 8 9]
x[4:7]     # [4 5 6]
x[::2]     # [0 2 4 6 8]
x[5::-2]   # [5 3 1]   ← 5부터 거꾸로 2칸씩 (자주 틀림)
```

슬라이싱 기본값: `start=0` , `stop=차원 크기` , `step=1`

모양 바꾸기와 전치

```
x = np.arange(12)
x.reshape(2, 6)      # 2행 6열
x.reshape(2, 2, 3)   # 3차원 (2, 2, 3)
x.reshape(5, 2)      # ValueError  (원소 12개 ≠ 5×2)

arr = np.arange(15).reshape(3, 5)
arr.T                # (5, 3)  행과 열 교환. transpose()도 같음
```

전치는 데이터를 **복사하지 않고** 모양만 바꾼 **뷰(view)**를 반환한다.

비교 배열 연결과 분할

함수	동작	(2, 3) 두 개 → 결과
np.concatenate([x, y])	기본 `axis=0` , 세로(행 방향)로 연결	(4, 3)
np.concatenate([x, y], axis=1)	가로(열 방향)로 연결	(2, 6)
np.vstack([x, y])	세로로 쌓기 = `axis=0`	(4, 3)
np.hstack([x, y])	가로로 쌓기 = `axis=1`	(2, 6)

`concatenate`는 배열들을 **리스트나 튜플로 묶어** 첫 번째 인자로 전달해야 한다.

```
z = np.arange(16).reshape(4, 4)
upper, lower = np.split(z, [2], axis=0)  # 2행 기준 위/아래 → 각각 (2, 4)
left, right = np.hsplit(z, [2])          # 2열 기준 좌/우 → 각각 (4, 2)
# np.vsplit: 행 단위(axis=0), np.hsplit: 열 단위(axis=1)
```

유니버설 함수 (ufunc)

- 원소 단위로 연산하는 함수. 벡터화되어 빠름
- **단항** (인자 1개): `np.abs`, `np.sqrt`, `np.exp` → `np.sqrt([1, 4, 9, 16])` → `[1. 2. 3. 4.]`
- **이항** (인자 2개): `np.add`, `np.maximum` → `np.add([1,2,3], [10,5,7])` → `[11 7 10]`

실습 브로드캐스팅 규칙

크기가 다른 배열끼리 연산할 때, 작은 배열이 큰 배열에 맞게 자동 확장되는 규칙

단계	규칙
1	차원 수가 다르면 작은 배열의 **앞쪽(왼쪽)을 1로 채움** → `(3,)`은 `(1, 3)`
2	어떤 차원에서 크기가 다르면 **크기가 1인 쪽**을 상대에 맞게 늘림
3	크기가 다르고 **어느 쪽도 1이 아니면** `ValueError`

```
# 예 1) (2, 3) + (3,)
a = np.array([[1, 2, 3], [4, 5, 6]]); b = np.array([10, 20, 30])
a + b                       # [[11 22 33] [14 25 36]]

# 예 2) 두 배열 모두 확장: (3, 1) + (3,) → (3, 1) + (1, 3) → (3, 3)
np.arange(3).reshape((3, 1)) + np.arange(3)
# [[0 1 2]
#  [1 2 3]
#  [2 3 4]]

# 예 3) 오류: (3, 2) + (3,) → 마지막 차원 2 ≠ 3, 둘 다 1 아님
np.ones((3, 2)) + np.arange(3)   # ValueError
```

4. 7주차 · 사용자 정의 함수와 제어 구문

함수 기본

- 함수는 입력값(매개변수)을 받아 실행하고 **하나의 결과값(객체)**을 반환. 입력값에만 영향을 받는 함수 = **순수(pure) 함수**
- 형식: `def 함수명(arglist):` + 들여쓴 `body`. `arglist`는 인자 이름과 기본값을 정의
- 필요한 인자 없이 호출하면 오류: `foo()` → `TypeError: missing 1 required positional argument`
- 정의한 함수는 **다른 함수 안에서 재사용** 가능

```
def square(x):
    return x * x

def sumsq(x):                    # 1부터 x까지 제곱의 합
    temp = []
    for i in range(1, x + 1):
        temp.append(square(i))   # square() 재사용
    return sum(temp)
```

```
sumsq(10)   # 385
```

비교 익명 함수 `lambda`

- `lambda 매개변수: 표현식` . **이름 없이** 정의하고, `return` 을 쓰지 않아도 **표현식 결과가 반환**됨
- 다른 함수의 **인자로** 많이 사용: 정렬(`sort` 의 `key`), `map` , `filter`

```
(lambda x, y: x + y)(10, 20)   # 30  (add(10, 20)과 같음)

students = [("Kim", 22), ("Lee", 19), ("Park", 25)]
students.sort(key=lambda x: x[1])   # 나이(두 번째 원소) 기준 정렬
# [('Lee', 19), ('Kim', 22), ('Park', 25)]
```

실습 인자 전달 방식

```
def menu(x1, x2, x3):
    return {"a": x1, "b": x2, "c": x3}

menu(1, 2, 3)                   # 위치 인자: 순서대로 → {'a': 1, 'b': 2, 'c': 3}
menu(2, 1, 3)                   # 순서가 바뀌면 결과도 바뀜 → {'a': 2, 'b': 1, 'c': 3}
menu(x3="x3", x2="x2", x1="x1") # 키워드 인자: 이름으로 연결, 순서 무관
menu("x1", x3="x3", x2="x2")    # 혼합: 위치 인자가 먼저 → OK
```

함정 위치 인자는 반드시 키워드 인자보다 앞에

```
menu(x3="딸기", x2="바나나", "사과")    # SyntaxError: positional argument follows keyword argument
menu("사과", x1="배", x3="딸기")        # TypeError: x1에 값이 두 번 들어감
menu(y1="사과", x2="바나나", x3="딸기") # TypeError: 없는 매개변수 이름
```

`SyntaxError` 는 문법 단계에서 막혀서 셀 전체가 실행되지 않고, `TypeError` 는 실행 중에 발생한다.

기본값, `*args` , `**kwargs`

```
def menu(x1, x2, x3=3):          # 기본값 지정
    return {"a": x1, "b": x2, "c": x3}
menu(1, 2)       # {'a': 1, 'b': 2, 'c': 3}  x3 생략 → 기본값
menu(1, 2, 4)    # {'a': 1, 'b': 2, 'c': 4}  입력하면 입력값 사용

def sumsq(*args):                # 위치 인자들을 튜플로 묶음
    result = 0
    for i in args:               # args = (1, 2, 3, 4, 5)
        result += i
    return result
sumsq(1, 2, 3, 4, 5)   # 15   ← 이름은 sumsq지만 제곱하지 않고 더하기만!
sumsq()                # 0    args = ()

def print_kwargs(**kwargs):      # 키워드 인자들을 딕셔너리로 묶음
    print("Keyword arguments:", kwargs)
print_kwargs(a=1, b=2, c=3)      # Keyword arguments: {'a': 1, 'b': 2, 'c': 3}
```

	`*args` (별 1개)	`**kwargs` (별 2개)
모으는 것	위치 인자	키워드 인자
묶는 형태	**튜플**	**딕셔너리** (인자 이름 = 키)

제어 구문과 연산자

- 코드는 기본적으로 위→아래, 왼쪽→오른쪽 순서로 실행. 제어 구문으로 **순차·선택·반복** 구조를 조합
- 조건문(선택): `if` - `else` · 반복문: `for`, `while`, `break`, `continue`
- 비교: `<` `<=` `>` `>=` `==` `!=` · 논리: `or` `and` `not` · 포함: `in`
- `if` - `else`는 둘 중 **하나만** 실행. `else`가 필요 없으면 `if`만 단독 사용 가능(단순 조건문)

실습 `is_even()` 세 가지 버전

```
# 버전 1: True / False 반환
def is_even(x):
    if x % 2 == 0:
        return True
    else:
        return False
is_even(5)      # False

# 버전 2: 직접 출력 → 정수가 아니면 잘못된 결과
def is_even(x):
    if x % 2 == 0:
        print(x, "is an even number")
    else:
        print(x, "is an odd number")
is_even(3.5)    # 3.5 is an odd number   ← 오류가 아니라 '잘못된 출력'

# 버전 3: 조건문 중첩으로 개선
def is_even(x):
    if x % 1 == 0 and x > 0:       # 정수이면서 양수일 때만
        if x % 2 == 0:
            print(x, "is an even number")
        else:
            print(x, "is an odd number")
    else:
        print("NA")
is_even(3.5)    # NA
is_even(0)      # NA  ← 0은 짝수인데 x > 0 조건 때문에 NA
is_even(-4)     # NA  ← 음수 짝수도 NA (개선 버전의 한계)
```

실습 `for` 반복문

- `for 변수 in 리스트/튜플/문자열:` → 길이만큼 반복하며 원소가 **하나씩 순서대로** 변수에 들어감
- 튜플이 담긴 리스트 `[(1, 2), (3, 4)]`를 돌면 **튜플 단위**로 처리
- `range(1, 8)` → 1~7 (끝값 미포함). `list(range(1, 8))[::-1]` → 7부터 1까지 역순

```
def sum_num(x):
    result = 0
    for i in range(x + 1):       # 0부터 x까지
        result = result + i
    print("Total:", result)
sum_num(10)    # Total: 55
```

```
# 강의에는 "벡터 x의 합"이라고 쓰여 있지만 실제로는 0~x 정수의 합.
# 또 return이 아니라 print라서 반환값은 None
```

비교 **`continue` VS `break`**

```
for i in range(1, 11):
    if i == 3:
        continue     # 이번 반복만 건너뛰고 다음으로
    elif i == 7:
        break        # 반복문 전체 종료
    print(i)
# 출력: 1 2 4 5 6
```

`zip()`, `enumerate()`, `while`

```
for name, score in zip(["Tom", "Jane", "Mike"], [90, 85, 95]):
    print(name, "scored", score)          # 같은 인덱스끼리 묶음
list(zip([1, 2, 3], ["a", "b"]))          # [(1, 'a'), (2, 'b')]  가장 짧은 쪽에 맞춤

for i, fruit in enumerate(["apple", "banana", "cherry"]):
    print(i, fruit)                       # 0 apple / 1 banana / 2 cherry

count = 1
while count <= 5:      # 조건이 참인 동안 반복
    print(count)
    count += 1          # 빼먹으면 무한 반복
```

`for` 는 **정해진 횟수**만큼, `while` 은 **조건이 참인 동안** 반복한다.

5. 실습 3 정리 (함수 작성)

1번: 온도 변환 함수

```
def fahr2celsius(temp):                 # 문제에서 주어진 함수
    celsius = (temp - 32) * (5 / 9)
    return celsius

def fahr2kelvin(temp):                  # (1) 기존 함수 재사용
    kelvin = fahr2celsius(temp) + 273.15
    return kelvin

def celsius2fahr(temp):                 # (2)
    fahr = (temp * 1.8) + 32
    return fahr

def cel2FK(temp):                       # (3) 딕셔너리 반환
    fahr = celsius2fahr(temp)
    kelvin = temp + 273.15
    return {"fahrenheit": fahr, "kelvin": kelvin}

fahr2kelvin(32)    # 273.15
celsius2fahr(100)  # 212.0
cel2FK(0)          # {'fahrenheit': 32.0, 'kelvin': 273.15}
```

포인트: 함수 안에서 다른 함수 호출(재사용) · 딕셔너리 반환 · 켈빈은 섭씨에서 바로 `+ 273.15` (화씨를 거쳐 돌아가면 -40°C에서 `233.14999999999998` 같은 오차)

2번: 생일 문제 `bday(n)`

n명 중 최소 두 명의 생일이 같을 확률 = 1 − (365/365 × 364/365 × ⋯ × (365−n+1)/365)

```
import math


def bday(n):
    nums = []
    for i in range(n):                  # i = 0, 1, ..., n-1 → n번
        nums.append((365 - i) / 365)
    return 1 - math.prod(nums)          # 리스트 원소를 모두 곱함

bday(2)   # 0.002739726027397249
bday(3)   # 0.008204165884781345
```

함정 작성 중 나왔던 실수 3가지 (오류 찾기 문제로 그대로 나올 수 있음)

```
for i in n - 1:          # TypeError: 'int' object is not iterable → range()로 감싸야 함
for i in range(n - 1):   # 반복이 한 번 부족 → bday(2)가 0이 됨. range(n)이 맞음
nums = []
nums[i] = 값              # IndexError: 빈 리스트에는 자리가 없음 → append() 사용
```

PEP 8 체크 (실습 채점 항목)

- 최상위 함수 정의 앞뒤로 **빈 줄 2개** (`import` 바로 뒤에 `def` 가 오면 그 사이도 2개)
- 연산자 앞뒤 공백: `365 - i` (O), `365-i` (X)
- 딕셔너리 콜론: `{"a": 1}` (O), `{"a" : 1}` (X)
- 들여쓰기는 공백 4칸, 제출 전 **Restart & Run All**

6. 강의 자료 속 함정 (코드와 설명이 다른 곳)

코드를 읽고 검토하는 시험이라면 "주석·라벨이 아니라 **실제 코드**를 보고 판단"하는 연습이 필요합니다.

위치	설명·라벨	실제 코드와 결과
6주차 `zeros`	라벨 `zeros((3, 6))`	`np.zeros(10)` → 1차원 0이 10개
6주차 `reshape`	주석 "(3x4)", "(3x2x2)"	`reshape(2, 6)` , `reshape(2, 2, 3)` → 실제 모양은 (2, 6), (2, 2, 3)
7주차 `is_even(3.5)`	주석 "오류"	에러 없이 `3.5 is an odd number` 출력 (잘못된 결과)
7주차 개선 `is_even`	"문제점 제거"	조건 `x > 0` 때문에 0, -4 같은 짝수도 `NA`
7주차 `sum_num(x)`	"벡터 x의 합"	0부터 x까지 정수의 합. `print` 만 하고 반환값은 `None`
7주차 `sumsq(*args)`	이름은 제곱의 합	제곱 없이 더하기만 → `sumsq(1, 2, 3, 4, 5)` = 15 (제곱합이면 55)
4~5주차 딕셔너리	`updated()` 함수	실제 메서드 이름은 `update()`

7. 시험 직전 체크리스트

1. `/` 는 항상 실수(`4 / 2` → `2.0`), `//` 는 몫, `int(98.7)` 은 `98` (버림)
2. 문자열·튜플은 수정 불가 → `TypeError` . 문자열은 `replace()` 나 슬라이싱으로 새로 만듦
3. `find()` 는 없으면 `-1` , 리스트 `index()` 는 없으면 `ValueError`
4. 삭제: `del` (위치) / `remove()` (값, 첫 번째만) / `pop()` (삭제 + 반환)
5. `lst.sort()` 의 반환값은 `None` , `sorted()` 는 새 리스트
6. `(70,)` 은 튜플, `(70)` 은 정수 · `{}` 는 빈 딕셔너리, 빈 집합은 `set()`
7. 딕셔너리 없는 키: `d["k"]` → `KeyError` , `d.get("k")` → `None`
8. NumPy `zeros` / `ones` 기본 타입은 `float64` , `size` = shape의 곱
9. `x[5::-2]` → `[5 3 1]` , `reshape` 는 원소 수가 맞아야 함(아니면 `ValueError`)
10. 브로드캐스팅: 왼쪽에 1 채우기 → 같거나 1이면 OK → 아니면 `ValueError`
11. 함수 호출 시 위치 인자가 키워드 인자보다 먼저 (아니면 `SyntaxError`)
12. `*args` → 튜플, `**kwargs` → 딕셔너리, `lambda` 는 이름 없는 함수로 표현식 결과를 반환
13. `continue` 는 이번 반복만 건너뜀, `break` 는 반복 전체 종료
14. `range(n)` 은 0~n-1, `zip()` 은 가장 짧은 쪽에 맞춤

원래 요약본에서 뺀 내용 (강의 PPT·노트북에 없음): NumPy 슬라이싱의 View/Copy와 `.copy()` , 변수 스코프(LEGB)와 `global` , `for-else` , `reshape(-1, n)` , `enumerate(start=1)` , 여러 값 반환(`return a, b`), 기본값 매개변수 순서 규칙, "오답률 80%" 같은 근거 없는 문구. 강의에 없는 내용은 출제 가능성이 낮지만, 교수님이 수업 중 따로 언급하셨다면 다시 챙겨 보세요.